МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»

Курсовая работа

по курсу Операционные системы

Выполнил: К. Р. Шестаков
Группа: М8О-207БВ-24
Преподаватель: Е. С. Миронов
Дата: 22.04.2026

Москва, 2026

1 Условие

Цель работы:

1. реализовать два пользовательских аллокатора памяти, работающих поверх заранее выделенного буфера;

2. сравнить их по скорости выделения памяти, скорости освобождения и эффективности использования памяти на одинаковых сценариях нагрузки.

Задание: спроектировать общий интерфейс распределителей памяти, реализовать аллокатор на основе списка свободных блоков со стратегией `first fit` и аллокатор с блоками размера 2^n, подготовить единый тестовый стенд, выполнить серию измерений и сделать выводы по полученным результатам.
Вариант: реализовать и сравнить пользовательский аллокатор со списком свободных блоков и аллокатор с блоками размера 2^n. Исследование проводится на собственных реализациях распределителей памяти без обращения к операционной системе во время работы: оба варианта используют заранее выделенный непрерывный буфер.

2 Метод решения

2.1 Общий подход

Оба аллокатора работают не с системным менеджером памяти процесса напрямую, а внутри заранее подготовленного массива байтов. Такой подход изолирует исследуемую логику от внешних факторов и позволяет сравнивать именно алгоритмы распределения памяти, а не поведение стандартного аллокатора операционной системы.

Снаружи обе реализации скрыты за общей структурой `Allocator`. В ней хранится тип аллокатора и указатель на внутреннее состояние. Благодаря этому тестовый стенд использует единый интерфейс и может запускать идентичные сценарии для обоих вариантов без изменения управляющего кода.

2.2 Аллокатор на основе списка свободных блоков

В первом варианте память представляется как двусвязный список блоков. Каждый блок содержит служебный заголовок с размером полезной области, флагом занятости и ссылками на соседние блоки. После инициализации весь буфер считается одним большим свободным блоком.

При вызове `alloc` аллокатор последовательно проходит по списку слева направо и выбирает первый свободный блок достаточного размера. Если выбранный блок заметно больше требуемого размера, он разделяется на две части: первая передаётся пользователю, вторая остаётся свободной. При освобождении памяти блок помечается как свободный и затем при возможности объединяется с соседними свободными блоками.

Преимущество такого подхода состоит в более экономном использовании памяти, поскольку размер выделенного блока близок к запрошенному. Основной недостаток — зависимость времени поиска от длины и текущей фрагментированности списка свободных блоков.

2.3 Аллокатор с блоками размера 2^n

Второй вариант построен по идее «buddy allocator». Вся память разбивается на блоки, размеры которых равны степеням двойки. Для каждого порядка поддерживается собственный список свободных блоков.

Если требуется блок размера `S`, аллокатор сначала учитывает размер служебного заголовка, затем выбирает минимальный блок 2^n, в который запрос помещается. Если свободного блока нужного порядка нет, берётся более крупный блок и делится пополам до тех пор, пока не будет получен блок подходящего размера.

При освобождении памяти вычисляется «парный» блок того же порядка. Если он тоже свободен, оба блока объединяются в более крупный. Такое слияние продолжается, пока это возможно. Достоинство этого подхода — стабильное и быстрое время выделения. Недостаток — внутренняя фрагментация, возникающая из-за округления запроса вверх до степени двойки.

2.4 Критерии сравнения

В работе сравниваются четыре показателя:

1. фактор использования памяти;
2. скорость выделения памяти;
3. скорость освобождения памяти;
4. простота использования и сопровождения реализации.

Под фактором использования памяти понимается отношение полезно запрошенной памяти к объёму, который аллокатор реально занял внутри буфера:

$$U = \frac{\text{requested_bytes}}{\text{reserved_bytes}}.$$

Чем ближе значение U к единице, тем меньше потери на служебные заголовки и внутреннюю фрагментацию.

3 Описание программы

3.1 Структура проекта

Проект разделён на несколько файлов:

- `src/allocator.h` — общий интерфейс распределителей памяти и структура статистики;
- `src/allocator.cpp` — реализация двух аллокаторов и вспомогательных функций;
- `src/benchmark.cpp` — генерация сценариев нагрузки, запуск измерений и вывод результатов;
- `CMakeLists.txt` — конфигурация сборки проекта;
- `report/report.md` — исходный текст отчёта в Markdown;
- `report/report.tex` — оформленная версия отчёта в формате LaTeX.

3.2 Внешний интерфейс

Обе реализации используют единый публичный интерфейс:

```
Allocator* createFirstFitAllocator(void* realMemory, size_t memorySize);
Allocator* createPowerOfTwoAllocator(void* realMemory, size_t memorySize);
void destroyAllocator(Allocator* allocator);

void* alloc(Allocator* allocator, size_t blockSize);
void freeBlock(Allocator* allocator, void* block);

AllocatorStats getAllocatorStats(const Allocator* allocator);
size_t getAllocatedBlockFootprint(const Allocator* allocator, const void* block
    );
const char* getAllocatorName(const Allocator* allocator);
```

Листинг 1: Публичный интерфейс аллокаторов

Функция освобождения названа `freeBlock`, чтобы не пересекаться со стандартной функцией `free`. Дополнительная функция `getAllocatedBlockFootprint` используется только в тестовом стенде и позволяет корректно вычислять фактический размер блока, занятого внутри буфера.

3.3 Ключевые структуры данных

В реализации используются две разные внутренние схемы хранения состояния:

- `FirstFitBlockHeader` и `FirstFitState` для варианта со списком свободных блоков;
- `BuddyBlockHeader` и `BuddyState` для варианта с блоками размера 2^n.

В обоих случаях служебные заголовки размещаются непосредственно внутри рабочего буфера. Это позволяет измерять реальные накладные расходы на хранение метаданных и учитывать их в статистике.

3.4 Тестовый стенд

Файл `src/benchmark.cpp` описывает три сценария нагрузки:

1. **Мелкие блоки.** Запросы от `16` до `128` байт.
2. **Смешанная нагрузка.** Запросы от `32` до `4096` байт.
3. **Фрагментация.** Чередование маленьких и больших блоков с частичным освобождением и последующей случайной нагрузкой.

Для измерения времени используются стандартные средства библиотеки C++: `std::chrono::steady_clock`. Повторяемость сценариев обеспечивается фиксированными значениями генератора случайных чисел.

3.5 Используемые системные средства

Прямые системные вызовы в программе не используются. Это является частью замысла работы: исследуемые аллокаторы во время тестов не обращаются к операционной системе за дополнительной памятью и не зависят от поведения внешнего распределителя. Сборка выполняется через CMake и MSBuild, а выполнение — в обычном консольном процессе Windows.

3.6 Сборка и запуск

Сборка проекта выполняется командами:

```
cmake -S . -B build
cmake --build build --config Release
```

Запуск тестового стенда:

```
build\Release\memory_allocators.exe
```

4 Результаты работы

4.1 Сценарий тестирования

Измерения проводились для трёх сценариев нагрузки, описанных выше. Во всех случаях использовался буфер объёмом `1 048 576` байт. Каждый сценарий повторялся 5 раз, после чего вычислялись средние значения времени `alloc` и `freeBlock`, среднего фактора использования памяти и его пикового значения.

Сборка выполнялась в режиме `Release` компилятором `MSVC 2022`. Такой режим выбран для того, чтобы сравнение отражало поведение реализаций без отладочных накладных расходов.

4.2 Фрагмент фактического вывода программы

Ниже приведён лог одного из запусков программы, полученный после сборки текущей версии проекта:

```
Сравнение аллокаторов памяти
Размер тестового буфера: 1048576 байт
Количество прогонов каждого сценария: 5

=== Мелкие блоки ===
Случайные блоки размером от 16 до 128 байт,умеренное число
одновременно живых объектов.
Список свободных блоков:
успешных выделений: 25253
отказов: 0
alloc,нс: 546.75
free,нс: 19.30
средн. использование: 0.63
пик: 0.79
Блоки 2^n:
успешных выделений: 25253
отказов: 0
```

```
alloc,нс: 24.82
free,нс: 21.91
средн. использование: 0.47
пик: 0.73

=== Смешанная нагрузка ===
Блоки размером от 32 до 4096 байт,нагрузка похожа на реальную
программу общего назначения.
Список свободных блоков:
успешных выделений: 25126
отказов: 0
alloc,нс: 368.16
free,нс: 25.81
средн. использование: 0.98
пик: 0.99
Блоки 2^n:
успешных выделений: 25126
отказов: 0
alloc,нс: 29.67
free,нс: 24.64
средн. использование: 0.73
пик: 0.84

=== Фрагментация ===
Чередование маленьких и больших блоков с частичным освобождением
для создания разрывов в памяти.
Список свободных блоков:
успешных выделений: 6351
отказов: 5247
alloc,нс: 1833.77
free,нс: 29.41
средн. использование: 0.97
пик: 0.98
Блоки 2^n:
успешных выделений: 7942
отказов: 3656
alloc,нс: 28.81
free,нс: 26.60
средн. использование: 0.54
пик: 0.59
```

4.3 Сводные результаты

Сценарий 1. Мелкие блоки

Аллокатор	Усп. выд.	Отказы	Alloc, нс	Free, нс	Средн. U	Пик. U
Список свободных блоков	25253	0	546.75	19.30	0.63	0.79
Блоки размера 2^n	25253	0	24.82	21.91	0.47	0.73

На мелких блоках аллокатор с блоками размера 2^n заметно быстрее выполняет выделение памяти. Разница почти на порядок определяется тем, что выбор подходящего блока идёт по готовым спискам фиксированных порядков. При этом вариант

`first fit` лучше использует память, так как не требует агрессивного округления размера запроса вверх.

Сценарий 2. Смешанная нагрузка

Аллокатор	Усп. выд.	Отказы	Alloc, нс	Free, нс	Средн. U	Пик. U
Список свободных блоков	25126	0	368.16	25.81	0.98	0.99
Блоки размера 2^n	25126	0	29.67	24.64	0.73	0.84

На смешанной нагрузке соотношение сохраняется. Аллокатор с блоками размера 2^n остаётся быстрее, однако теряет в эффективности использования памяти. Для `first fit` особенно хорошо видно, что почти весь фактически занятый объём идёт в полезные данные.

Сценарий 3. Фрагментация

Аллокатор	Усп. выд.	Отказы	Alloc, нс	Free, нс	Средн. U	Пик. U
Список свободных блоков	6351	5247	1833.77	29.41	0.97	0.98
Блоки размера 2^n	7942	3656	28.81	26.60	0.54	0.59

При фрагментации различие между подходами становится ещё сильнее. Аллокатор `first fit` по-прежнему экономно расходует память, но резко проигрывает по времени выделения, поскольку поиск первого подходящего блока выполняется на длинной и фрагментированной цепочке. Вариант с блоками размера 2^n сохраняет почти постоянное время `alloc` и показывает меньше отказов.

4.4 Итог сравнения

По результатам измерений можно сделать следующие выводы:

- аллокатор `first fit` выигрывает по эффективности использования памяти;
- аллокатор с блоками размера 2^n выигрывает по скорости выделения памяти;
- по скорости освобождения оба подхода близки, но вариант 2^n остаётся немного стабильнее на сложных сценариях;
- с точки зрения объяснимости и сопровождения реализация списка свободных блоков проще, чем реализация «buddy allocator».

5 Выводы

В ходе работы были реализованы два пользовательских аллокатора памяти поверх заранее выделенного буфера и проведено их сравнение на нескольких сценариях нагрузки. Подход на основе списка свободных блоков со стратегией `first fit` показал более экономный расход памяти и высокую эффективность при умеренной фрагментации.

Аллокатор с блоками размера 2^n продемонстрировал существенно более высокую скорость выделения памяти и лучшую устойчивость к фрагментации, однако платой за это стала внутренняя фрагментация и менее высокий коэффициент использования памяти.

Следовательно, однозначно лучшего варианта здесь нет. Если в прикладной задаче важнее минимизировать потери памяти, предпочтителен `first fit`. Если критична скорость выделения и предсказуемость времени работы даже в неблагоприятных состояниях памяти, практичнее использовать аллокатор с блоками размера 2^n.

6 Исходный код программы

6.1 Файл src/allocator.h

```
#pragma once

#include <cstddef>

enum class AllocatorKind {
    FirstFit,
    PowerOfTwo
};

struct AllocatorStats {
    size_t managed_memory = 0;
    size_t used_memory = 0;
    size_t free_memory = 0;
    size_t largest_free_block = 0;
    size_t allocation_count = 0;
};

struct Allocator {
    AllocatorKind kind = AllocatorKind::FirstFit;
    void* state = nullptr;
};

Allocator* createFirstFitAllocator(void* realMemory, size_t memorySize);
Allocator* createPowerOfTwoAllocator(void* realMemory, size_t memorySize);
void destroyAllocator(Allocator* allocator);

void* alloc(Allocator* allocator, size_t blockSize);
void freeBlock(Allocator* allocator, void* block);

AllocatorStats getAllocatorStats(const Allocator* allocator);
size_t getAllocatedBlockFootprint(const Allocator* allocator, const void* block);
const char* getAllocatorName(const Allocator* allocator);
```

Листинг 2: src/allocator.h

6.2 Файл src/allocator.cpp

```
#include "allocator.h"

#include <algorithm>
#include <cstdint>
#include <cstring>
#include <new>
#include <vector>

namespace {

constexpr size_t kAlignment = alignof(std::max_align_t);

size_t alignUp(size_t value, size_t alignment) {
    return (value + alignment - 1) / alignment * alignment;
}
```

```

std::byte* alignPointer(void* memory) {
    const auto address = reinterpret_cast<std::uintptr_t>(memory);
    const auto aligned = alignUp(address, kAlignment);
    return reinterpret_cast<std::byte*>(aligned);
}

struct MemoryRange {
    std::byte* begin = nullptr;
    size_t size = 0;
};

MemoryRange normalizeRange(void* memory, size_t memorySize) {
    if (memory == nullptr || memorySize == 0) {
        return {};
    }

    std::byte* alignedBegin = alignPointer(memory);
    const auto shift = static_cast<size_t>(alignedBegin - static_cast<std::byte*>(memory));

    if (shift >= memorySize) {
        return {};
    }

    return {alignedBegin, memorySize - shift};
}

size_t floorLog2(size_t value) {
    size_t result = 0;
    while ((size_t(1) << (result + 1)) <= value) {
        ++result;
    }
    return result;
}

size_t ceilLog2(size_t value) {
    size_t result = 0;
    size_t power = 1;
    while (power < value) {
        power <<= 1;
        ++result;
    }
    return result;
}

size_t highestPowerOfTwo(size_t value) {
    if (value == 0) {
        return 0;
    }
    return size_t(1) << floorLog2(value);
}

struct FirstFitBlockHeader {
    size_t size = 0;
    bool isFree = true;
    FirstFitBlockHeader* next = nullptr;
    FirstFitBlockHeader* prev = nullptr;
```

```
};

struct FirstFitState {
    std::byte* memoryBegin = nullptr;
    size_t memorySize = 0;
    FirstFitBlockHeader* head = nullptr;
};

struct BuddyBlockHeader {
    size_t order = 0;
    bool isFree = true;
    BuddyBlockHeader* next = nullptr;
    BuddyBlockHeader* prev = nullptr;
};

struct BuddyState {
    std::byte* memoryBegin = nullptr;
    size_t memorySize = 0;
    size_t minOrder = 0;
    size_t maxOrder = 0;
    std::vector<BuddyBlockHeader*> freeLists;
};

void splitFirstFitBlock(FirstFitBlockHeader* block, size_t requestedSize) {
    const size_t headerSize = sizeof(FirstFitBlockHeader);
    const size_t remainder = block->size - requestedSize;

    if (remainder < headerSize + kAlignment) {
        return;
    }

    auto* nextBlock = reinterpret_cast<FirstFitBlockHeader*>(
        reinterpret_cast<std::byte*>(block) + headerSize + requestedSize
    );

    nextBlock->size = remainder - headerSize;
    nextBlock->isFree = true;
    nextBlock->prev = block;
    nextBlock->next = block->next;

    if (block->next != nullptr) {
        block->next->prev = nextBlock;
    }

    block->size = requestedSize;
    block->next = nextBlock;
}

void mergeWithNext(FirstFitBlockHeader* block) {
    while (block != nullptr && block->next != nullptr && block->next->isFree) {
        FirstFitBlockHeader* nextBlock = block->next;
        block->size += sizeof(FirstFitBlockHeader) + nextBlock->size;
        block->next = nextBlock->next;

        if (block->next != nullptr) {
            block->next->prev = block;
        }
    }
```

```
}

void buddyAddFreeBlock(BuddyState* state, BuddyBlockHeader* block) {
    const size_t order = block->order;
    block->isFree = true;
    block->prev = nullptr;
    block->next = state->freeLists[order];

    if (state->freeLists[order] != nullptr) {
        state->freeLists[order]->prev = block;
    }

    state->freeLists[order] = block;
}

void buddyRemoveFreeBlock(BuddyState* state, BuddyBlockHeader* block) {
    const size_t order = block->order;

    if (block->prev != nullptr) {
        block->prev->next = block->next;
    } else {
        state->freeLists[order] = block->next;
    }

    if (block->next != nullptr) {
        block->next->prev = block->prev;
    }

    block->prev = nullptr;
    block->next = nullptr;
    block->isFree = false;
}

size_t buddyBlockSize(size_t order) {
    return size_t(1) << order;
}

size_t buddyPayloadCapacity(size_t order) {
    const size_t blockSize = buddyBlockSize(order);
    if (blockSize <= sizeof(BuddyBlockHeader)) {
        return 0;
    }
    return blockSize - sizeof(BuddyBlockHeader);
}

AllocatorStats collectFirstFitStats(const FirstFitState* state) {
    AllocatorStats stats{};
    stats.managed_memory = state->memorySize;

    for (FirstFitBlockHeader* block = state->head; block != nullptr; block = block->
        next) {
        if (block->isFree) {
            stats.free_memory += block->size;
            stats.largest_free_block = std::max(stats.largest_free_block, block->size);
        } else {
            stats.used_memory += sizeof(FirstFitBlockHeader) + block->size;
            ++stats.allocation_count;
        }
```

```
    }

    return stats;
}

AllocatorStats collectBuddyStats(const BuddyState* state) {
    AllocatorStats stats{};
    stats.managed_memory = state->memorySize;

    size_t offset = 0;
    while (offset < state->memorySize) {
        auto* block = reinterpret_cast<BuddyBlockHeader*>(state->memoryBegin + offset);
        const size_t size = buddyBlockSize(block->order);

        if (block->isFree) {
            const size_t payload = buddyPayloadCapacity(block->order);
            stats.free_memory += payload;
            stats.largest_free_block = std::max(stats.largest_free_block, payload);
        } else {
            stats.used_memory += size;
            ++stats.allocation_count;
        }

        offset += size;
    }

    return stats;
}

void* allocateFirstFit(Allocator* allocator, size_t blockSize) {
    if (allocator == nullptr || blockSize == 0) {
        return nullptr;
    }

    auto* state = static_cast<FirstFitState*>(allocator->state);
    const size_t requestedSize = alignUp(blockSize, kAlignment);

    for (FirstFitBlockHeader* block = state->head; block != nullptr; block = block->
        next) {
        if (!block->isFree || block->size < requestedSize) {
            continue;
        }

        splitFirstFitBlock(block, requestedSize);
        block->isFree = false;
        return reinterpret_cast<std::byte*>(block) + sizeof(FirstFitBlockHeader);
    }

    return nullptr;
}

void freeFirstFit(Allocator* allocator, void* blockPointer) {
    if (allocator == nullptr || blockPointer == nullptr) {
        return;
    }

    auto* block = reinterpret_cast<FirstFitBlockHeader*>(
        static_cast<std::byte*>(blockPointer) - sizeof(FirstFitBlockHeader)
```

```
    );

    if (block->isFree) {
        return;
    }

    block->isFree = true;
    mergeWithNext(block);

    if (block->prev != nullptr && block->prev->isFree) {
        mergeWithNext(block->prev);
    }
}

void* allocateBuddy(Allocator* allocator, size_t blockSize) {
    if (allocator == nullptr || blockSize == 0) {
        return nullptr;
    }

    auto* state = static_cast<BuddyState*>(allocator->state);
    const size_t requested = blockSize + sizeof(BuddyBlockHeader);
    const size_t targetOrder = std::max(state->minOrder, ceilLog2(requested));

    if (targetOrder > state->maxOrder) {
        return nullptr;
    }

    size_t currentOrder = targetOrder;
    while (currentOrder <= state->maxOrder && state->freeLists[currentOrder] ==
        nullptr) {
        ++currentOrder;
    }

    if (currentOrder > state->maxOrder) {
        return nullptr;
    }

    BuddyBlockHeader* block = state->freeLists[currentOrder];
    buddyRemoveFreeBlock(state, block);

    while (currentOrder > targetOrder) {
        --currentOrder;
        const size_t splitSize = buddyBlockSize(currentOrder);

        auto* rightBuddy = reinterpret_cast<BuddyBlockHeader*>(
            reinterpret_cast<std::byte*>(block) + splitSize
        );

        block->order = currentOrder;
        block->isFree = true;
        block->next = nullptr;
        block->prev = nullptr;

        rightBuddy->order = currentOrder;
        rightBuddy->isFree = true;
        rightBuddy->next = nullptr;
        rightBuddy->prev = nullptr;

```

```
        buddyAddFreeBlock(state, rightBuddy);
    }

    block->order = targetOrder;
    block->isFree = false;
    block->next = nullptr;
    block->prev = nullptr;

    return reinterpret_cast<std::byte*>(block) + sizeof(BuddyBlockHeader);
}

void freeBuddy(Allocator* allocator, void* blockPointer) {
    if (allocator == nullptr || blockPointer == nullptr) {
        return;
    }

    auto* state = static_cast<BuddyState*>(allocator->state);
    auto* block = reinterpret_cast<BuddyBlockHeader*>(
        static_cast<std::byte*>(blockPointer) - sizeof(BuddyBlockHeader)
    );

    if (block->isFree) {
        return;
    }

    std::byte* blockAddress = reinterpret_cast<std::byte*>(block);
    size_t order = block->order;

    while (order < state->maxOrder) {
        const size_t blockSize = buddyBlockSize(order);
        const size_t offset = static_cast<size_t>(blockAddress - state->memoryBegin);
        const size_t buddyOffset = offset ^ blockSize;

        if (buddyOffset + blockSize > state->memorySize) {
            break;
        }

        auto* buddy = reinterpret_cast<BuddyBlockHeader*>(state->memoryBegin +
            buddyOffset);
        if (!buddy->isFree || buddy->order != order) {
            break;
        }

        buddyRemoveFreeBlock(state, buddy);

        if (buddy < block) {
            block = buddy;
            blockAddress = reinterpret_cast<std::byte*>(block);
        }

        ++order;
        block->order = order;
        block->isFree = false;
    }

    buddyAddFreeBlock(state, block);
}

```

```
}

Allocator* createFirstFitAllocator(void* realMemory, size_t memorySize) {
    const MemoryRange range = normalizeRange(realMemory, memorySize);

    if (range.size < sizeof(FirstFitBlockHeader) + kAlignment) {
        return nullptr;
    }

    auto* allocator = new (std::nothrow) Allocator{};
    auto* state = new (std::nothrow) FirstFitState{};

    if (allocator == nullptr || state == nullptr) {
        delete allocator;
        delete state;
        return nullptr;
    }

    state->memoryBegin = range.begin;
    state->memorySize = range.size;
    state->head = reinterpret_cast<FirstFitBlockHeader*>(range.begin);
    state->head->size = range.size - sizeof(FirstFitBlockHeader);
    state->head->isFree = true;
    state->head->next = nullptr;
    state->head->prev = nullptr;

    allocator->kind = AllocatorKind::FirstFit;
    allocator->state = state;
    return allocator;
}

Allocator* createPowerOfTwoAllocator(void* realMemory, size_t memorySize) {
    const MemoryRange range = normalizeRange(realMemory, memorySize);
    const size_t managedSize = highestPowerOfTwo(range.size);
    const size_t minimumBlockSize = std::max(alignUp(sizeof(BuddyBlockHeader),
        kAlignment), size_t(32));

    if (managedSize < minimumBlockSize) {
        return nullptr;
    }

    auto* allocator = new (std::nothrow) Allocator{};
    auto* state = new (std::nothrow) BuddyState{};

    if (allocator == nullptr || state == nullptr) {
        delete allocator;
        delete state;
        return nullptr;
    }

    state->memoryBegin = range.begin;
    state->memorySize = managedSize;
    state->minOrder = ceilLog2(minimumBlockSize);
    state->maxOrder = floorLog2(managedSize);
    state->freeLists.assign(state->maxOrder + 1, nullptr);

    auto* initialBlock = reinterpret_cast<BuddyBlockHeader*>(state->memoryBegin);
    initialBlock->order = state->maxOrder;
```

```
    initialBlock->isFree = true;
    initialBlock->next = nullptr;
    initialBlock->prev = nullptr;
    state->freeLists[state->maxOrder] = initialBlock;

    allocator->kind = AllocatorKind::PowerOfTwo;
    allocator->state = state;
    return allocator;
}

void destroyAllocator(Allocator* allocator) {
    if (allocator == nullptr) {
        return;
    }

    if (allocator->kind == AllocatorKind::FirstFit) {
        delete static_cast<FirstFitState*>(allocator->state);
    } else {
        delete static_cast<BuddyState*>(allocator->state);
    }

    delete allocator;
}

void* alloc(Allocator* allocator, size_t blockSize) {
    if (allocator == nullptr) {
        return nullptr;
    }

    if (allocator->kind == AllocatorKind::FirstFit) {
        return allocateFirstFit(allocator, blockSize);
    }

    return allocateBuddy(allocator, blockSize);
}

void freeBlock(Allocator* allocator, void* block) {
    if (allocator == nullptr || block == nullptr) {
        return;
    }

    if (allocator->kind == AllocatorKind::FirstFit) {
        freeFirstFit(allocator, block);
        return;
    }

    freeBuddy(allocator, block);
}

AllocatorStats getAllocatorStats(const Allocator* allocator) {
    if (allocator == nullptr) {
        return {};
    }

    if (allocator->kind == AllocatorKind::FirstFit) {
        return collectFirstFitStats(static_cast<const FirstFitState*>(allocator->state)
            );
    }
```

```

    return collectBuddyStats(static_cast<const BuddyState*>(allocator->state));
}

size_t getAllocatedBlockFootprint(const Allocator* allocator, const void* blockPointer
    ) {
    if (allocator == nullptr || blockPointer == nullptr) {
        return 0;
    }

    if (allocator->kind == AllocatorKind::FirstFit) {
        const auto* block = reinterpret_cast<const FirstFitBlockHeader*>(
            static_cast<const std::byte*>(blockPointer) - sizeof(FirstFitBlockHeader)
        );
        return sizeof(FirstFitBlockHeader) + block->size;
    }

    const auto* block = reinterpret_cast<const BuddyBlockHeader*>(
        static_cast<const std::byte*>(blockPointer) - sizeof(BuddyBlockHeader)
    );
    return buddyBlockSize(block->order);
}

const char* getAllocatorName(const Allocator* allocator) {
    if (allocator == nullptr) {
        return "\u043d\u0435\u0438\u0437\u0432\u0435\u0441\u0442\u043d\u043e";
    }

    return allocator->kind == AllocatorKind::FirstFit
        ? "\u0421\u043f\u0438\u0441\u043e\u043a \u0441\u0432\u043e\u0431\u043e\u0434\
            u043d\u044b\u0445 \u0431\u043b\u043e\u043a\u043e\u0432"
        : "\u0411\u043b\u043e\u043a\u0438 2^n";
}
```

Листинг 3: src/allocator.cpp

6.3 Файл src/benchmark.cpp

```
#include "allocator.h"

#include <algorithm>
#include <chrono>
#include <cstddef>
#include <cstdint>
#include <iomanip>
#include <iostream>
#include <random>
#include <string>
#include <stdexcept>
#include <unordered_map>
#include <utility>
#include <vector>

struct Operation {
    enum class Type {
        Allocate,
```

```
        Free
    };

    Type type = Type::Allocate;
    size_t id = 0;
    size_t size = 0;
};

struct Scenario {
    std::string name;
    std::string description;
    std::vector<Operation> operations;
};

struct ActiveAllocation {
    void* pointer = nullptr;
    size_t requestedSize = 0;
    size_t reservedSize = 0;
};

struct ScenarioResult {
    std::string allocatorName;
    std::string scenarioName;
    size_t totalAllocationRequests = 0;
    size_t successfulAllocations = 0;
    size_t failedAllocations = 0;
    double averageAllocationNs = 0.0;
    double averageFreeNs = 0.0;
    double averageUtilization = 0.0;
    double peakUtilization = 0.0;
    size_t peakRequestedBytes = 0;
    size_t peakReservedBytes = 0;
};

std::vector<Operation> generateRandomOperations(
    size_t operationCount,
    size_t maxLiveBlocks,
    size_t minSize,
    size_t maxSize,
    double freeProbability,
    uint32_t seed
) {
    std::mt19937 generator(seed);
    std::uniform_int_distribution<size_t> sizeDistribution(minSize, maxSize);
    std::uniform_real_distribution<double> actionDistribution(0.0, 1.0);

    std::vector<size_t> activeIds;
    std::vector<Operation> operations;
    operations.reserve(operationCount + maxLiveBlocks);

    size_t nextId = 0;
    for (size_t i = 0; i < operationCount; ++i) {
        const bool mustAllocate = activeIds.empty();
        const bool canAllocateMore = activeIds.size() < maxLiveBlocks;
        const bool chooseAllocate = mustAllocate || (canAllocateMore &&
            actionDistribution(generator) > freeProbability);

        if (chooseAllocate) {
```

```
            const size_t id = nextId++;
            operations.push_back({Operation::Type::Allocate, id, sizeDistribution(generator)});
            activeIds.push_back(id);
        } else {
            std::uniform_int_distribution<size_t> indexDistribution(0, activeIds.size() - 1);
            const size_t index = indexDistribution(generator);
            operations.push_back({Operation::Type::Free, activeIds[index], 0});
            activeIds.erase(activeIds.begin() + static_cast<std::ptrdiff_t>(index));
        }
    }

    for (size_t id : activeIds) {
        operations.push_back({Operation::Type::Free, id, 0});
    }

    return operations;
}

Scenario buildSmallBlocksScenario() {
    return {
        "\u041c\u0435\u043b\u043a\u0438\u0435 \u0431\u043b\u043e\u043a\u0438",
        "\u0421\u043b\u0443\u0447\u0430\u0439\u043d\u044b\u0435 \u0431\u043b\u043e\u043a\u0438 \u0440\u0430\u0437\u043c\u0435\u0440\u043e\u043c \u043e\u0442 16 \u0434\u043e 128 \u0431\u0430\u0439\u0442, \u0443\u043c\u0435\u0440\u0435\u043d\u043d\u043e\u0435 \u0447\u0438\u0441\u043b\u043e \u043e\u0434\u043d\u043e\u0432\u0440\u0435\u043c\u0435\u043d\u043d\u043e \u0436\u0438\u0432\u044b\u0445 \u043e\u0431\u044a\u0435\u043a\u0442\u043e\u0432.",
        generateRandomOperations(50000, 512, 16, 128, 0.45, 42)
    };
}

Scenario buildMixedScenario() {
    return {
        "\u0421\u043c\u0435\u0448\u0430\u043d\u043d\u0430\u044f \u043d\u0430\u0433\u0440\u0443\u043a\u0430",
        "\u0411\u043b\u043e\u043a\u0438 \u0440\u0430\u0437\u043c\u0435\u0440\u043e\u043c \u043e\u0442 32 \u0434\u043e 4096 \u0431\u0430\u0439\u0442, \u043d\u0430\u0433\u0440\u0443\u043a\u0430 \u043f\u043e\u0445\u043e\u0436\u0430 \u043d\u0430 \u0440\u0435\u0430\u043b\u�ctu043d\u0443\u044e \u043f\u0440\u043e\u0433\u0440\u0430\u043c\u043c\u0443 \u043e\u0431\u0449\u0435\u0433\u043e \u043d\u0437\u043d\u0430\u0447\u0435\u043d\u0438\u044f.",
        generateRandomOperations(50000, 256, 32, 4096, 0.48, 1337)
    };
}

Scenario buildFragmentationScenario() {
    std::vector<Operation> operations;
    operations.reserve(25000);

    size_t nextId = 0;
    std::vector<size_t> smallIds;
    std::vector<size_t> largeIds;

    for (size_t i = 0; i < 2000; ++i) {
        const size_t smallId = nextId++;
        const size_t largeId = nextId++;
```

```
        operations.push_back({Operation::Type::Allocate, smallId, 64});
        operations.push_back({Operation::Type::Allocate, largeId, 2048});
        smallIds.push_back(smallId);
        largeIds.push_back(largeId);
    }

    for (size_t i = 0; i < smallIds.size(); i += 2) {
        operations.push_back({Operation::Type::Free, smallIds[i], 0});
    }
    for (size_t i = 0; i < largeIds.size(); i += 2) {
        operations.push_back({Operation::Type::Free, largeIds[i], 0});
    }

    auto randomPart = generateRandomOperations(15000, 300, 24, 3000, 0.5, 2024);
    for (Operation& operation : randomPart) {
        operation.id += nextId;
    }
    operations.insert(operations.end(), randomPart.begin(), randomPart.end());

    for (size_t i = 1; i < smallIds.size(); i += 2) {
        operations.push_back({Operation::Type::Free, smallIds[i], 0});
    }
    for (size_t i = 1; i < largeIds.size(); i += 2) {
        operations.push_back({Operation::Type::Free, largeIds[i], 0});
    }

    return {
        "\u0424\u0440\u0430\u0433\u043c\u0435\u043d\u0442\u0430\u0446\u0438\u044f",
        "\u0427\u0435\u0440\u0435\u0434\u043e\u0432\u0430\u043d\u0438\u0435 \u043c\
            u0430\u043b\u0435\u043d\u044c\u043a\u0438\u0445 \u0438 \u0431\u043e\u043b\
            u044c\u0448\u0438\u0445 \u0431\u043b\u043e\u043a\u043e\u0432 \u0441 \u0447\
            u0430\u0441\u0442\u0438\u0447\u043d\u044b\u043c \u043e\u0441\u0432\u043e\
            u0431\u043e\u0436\u0434\u0435\u043d\u0438\u0435\u043c \u0434\u043b\u044f \
            u0441\u043e\u0437\u0434\u0430\u043d\u0438\u044f \u0440\u0430\u0437\u0440\
            u044b\u0432\u043e\u0432 \u0432 \u043f\u0430\u043c\u044f\u0442\u0438.",
        std::move(operations)
    };
}

ScenarioResult runScenario(const Scenario& scenario, AllocatorKind kind, size_t
    bufferSize) {
    std::vector<std::byte> memory(bufferSize + 64);

    Allocator* allocator = nullptr;
    if (kind == AllocatorKind::FirstFit) {
        allocator = createFirstFitAllocator(memory.data(), memory.size());
    } else {
        allocator = createPowerOfTwoAllocator(memory.data(), memory.size());
    }

    if (allocator == nullptr) {
        throw std::runtime_error("\u041d\u0435 \u0443\u0434\u0430\u043b\u043e\u0441\
            u044c \u0441\u043e\u0437\u0434\u0430\u0442\u044c \u0430\u043b\u043b\u043e\
            u043a\u0430\u0442\u043e\u0440.");
    }

    ScenarioResult result;
    result.allocatorName = getAllocatorName(allocator);
```

```
    result.scenarioName = scenario.name;

    std::unordered_map<size_t, ActiveAllocation> active;
    active.reserve(2048);

    size_t currentRequestedBytes = 0;
    size_t currentReservedBytes = 0;
    double utilizationSum = 0.0;
    size_t utilizationSamples = 0;

    long long allocationTimeNs = 0;
    long long freeTimeNs = 0;
    size_t freeOperations = 0;

    for (const Operation& operation : scenario.operations) {
        if (operation.type == Operation::Type::Allocate) {
            ++result.totalAllocationRequests;

            const auto start = std::chrono::steady_clock::now();
            void* pointer = alloc(allocator, operation.size);
            const auto finish = std::chrono::steady_clock::now();

            allocationTimeNs += std::chrono::duration_cast<std::chrono::nanoseconds>(
                finish - start).count();

            if (pointer != nullptr) {
                const size_t reservedSize = getAllocatedBlockFootprint(allocator,
                    pointer);
                active[operation.id] = {pointer, operation.size, reservedSize};
                currentRequestedBytes += operation.size;
                currentReservedBytes += reservedSize;
                ++result.successfulAllocations;
            } else {
                ++result.failedAllocations;
            }
        } else {
            auto it = active.find(operation.id);
            if (it == active.end()) {
                continue;
            }

            const auto start = std::chrono::steady_clock::now();
            freeBlock(allocator, it->second.pointer);
            const auto finish = std::chrono::steady_clock::now();

            freeTimeNs += std::chrono::duration_cast<std::chrono::nanoseconds>(finish -
                 start).count();
            currentRequestedBytes -= it->second.requestedSize;
            currentReservedBytes -= it->second.reservedSize;
            active.erase(it);
            ++freeOperations;
        }

        const AllocatorStats stats = getAllocatorStats(allocator);
        if (currentRequestedBytes > 0 && currentReservedBytes > 0) {
            const double utilization = static_cast<double>(currentRequestedBytes) /
                static_cast<double>(currentReservedBytes);

```

```
            utilizationSum += utilization;
            ++utilizationSamples;
            result.peakUtilization = std::max(result.peakUtilization, utilization);
        }

        result.peakRequestedBytes = std::max(result.peakRequestedBytes, currentRequestedBytes);
        result.peakReservedBytes = std::max(result.peakReservedBytes, currentReservedBytes);
    }

    result.averageAllocationNs = result.totalAllocationRequests == 0
        ? 0.0
        : static_cast<double>(allocationTimeNs) / static_cast<double>(result.totalAllocationRequests);

    result.averageFreeNs = freeOperations == 0
        ? 0.0
        : static_cast<double>(freeTimeNs) / static_cast<double>(freeOperations);

    result.averageUtilization = utilizationSamples == 0
        ? 0.0
        : utilizationSum / static_cast<double>(utilizationSamples);

    destroyAllocator(allocator);
    return result;
}

ScenarioResult runScenarioRepeated(const Scenario& scenario, AllocatorKind kind, size_t bufferSize, size_t repetitions) {
    ScenarioResult accumulated;

    for (size_t i = 0; i < repetitions; ++i) {
        ScenarioResult current = runScenario(scenario, kind, bufferSize);

        if (i == 0) {
            accumulated.allocatorName = current.allocatorName;
            accumulated.scenarioName = current.scenarioName;
            accumulated.totalAllocationRequests = current.totalAllocationRequests;
            accumulated.successfulAllocations = current.successfulAllocations;
            accumulated.failedAllocations = current.failedAllocations;
            accumulated.peakRequestedBytes = current.peakRequestedBytes;
            accumulated.peakReservedBytes = current.peakReservedBytes;
        }

        accumulated.averageAllocationNs += current.averageAllocationNs;
        accumulated.averageFreeNs += current.averageFreeNs;
        accumulated.averageUtilization += current.averageUtilization;
        accumulated.peakUtilization += current.peakUtilization;
    }

    accumulated.averageAllocationNs /= static_cast<double>(repetitions);
    accumulated.averageFreeNs /= static_cast<double>(repetitions);
    accumulated.averageUtilization /= static_cast<double>(repetitions);
    accumulated.peakUtilization /= static_cast<double>(repetitions);

    return accumulated;
}
```

```

void printScenarioHeader(const Scenario& scenario) {
    std::cout << "\n=== " << scenario.name << " ===\n";
    std::cout << scenario.description << "\n";
}

void printResult(const ScenarioResult& result) {
    std::cout << result.allocatorName
              << " | \u0443\u0441\u043f\u0435\u0448\u043d\u044b\u0445 \u0432\u044b\u0434\u0435\u043b\u0435\u043d\u0438\u0439: " << std::setw(6) << result.successfulAllocations
              << " | \u043e\u0442\u043a\u0430\u0437\u043e\u0432: " << std::setw(6) << result.failedAllocations
              << " | alloc, \u043d\u0441: " << std::setw(10) << std::fixed << std::setprecision(2) << result.averageAllocationNs
              << " | free, \u043d\u0441: " << std::setw(10) << result.averageFreeNs
              << " | \u0441\u0440\u0435\u0434\u043d. \u0438\u0441\u043f\u043e\u043b\u044c\u0437\u043e\u0432\u0430\u043d\u0438\u0435: " << std::setw(6) << result.averageUtilization
              << " | \u043f\u0438\u043a: " << result.peakUtilization << "\n";
}

int main() {
    try {
        constexpr size_t bufferSize = 1 << 20;
        constexpr size_t repetitions = 5;

        std::vector<Scenario> scenarios;
        scenarios.push_back(buildSmallBlocksScenario());
        scenarios.push_back(buildMixedScenario());
        scenarios.push_back(buildFragmentationScenario());

        std::cout << "\u0421\u0440\u0430\u0432\u043d\u0435\u043d\u0438\u0435 \u0430\u043b\u043b\u043e\u043a\u0430\u0442\u043e\u0440\u043e\u0432 \u043f\u0430\u043c\u044f\u0442\u0438\n";
        std::cout << "\u0420\u0430\u0437\u043c\u0435\u0440 \u0442\u0435\u0441\u0442\u043e\u0432\u043e\u0433\u043e \u0431\u0443\u0444\u0� \u0430\u0439\u0442\n";
        std::cout << "\u041a\u043e\u043b\u0438\u0447\u0435\u0441\u0442\u0432\u043e \u043f\u0440\u043e\u0433\u043e\u043d\u043e\u0432 \u043a\u0430\u0436\u0434\u043e\u0433\u043e \u0441\u0446\u0435\u043d\u0430\u0440\u0438\u044f: " << repetitions << "\n";

        for (const Scenario& scenario : scenarios) {
            printScenarioHeader(scenario);
            const ScenarioResult firstFit = runScenarioRepeated(scenario, AllocatorKind::FirstFit, bufferSize, repetitions);
            const ScenarioResult powerOfTwo = runScenarioRepeated(scenario, AllocatorKind::PowerOfTwo, bufferSize, repetitions);

            printResult(firstFit);
            printResult(powerOfTwo);
        }
    } catch (const std::exception& exception) {
        std::cerr << "\u041e\u0448\u0438\u0431\u043a\u0430: " << exception.what() << "\n";
        return 1;
    }
```

```

    return 0;
}
```

Листинг 4: src/benchmark.cpp

6.4 Файл CMakeLists.txt

```
cmake_minimum_required(VERSION 3.16)
project(memory_allocators_coursework LANGUAGES CXX)

set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_CXX_EXTENSIONS OFF)

add_executable(memory_allocators
    src/allocator.cpp
    src/benchmark.cpp
)

target_include_directories(memory_allocators PRIVATE src)

if(MSVC)
    target_compile_options(memory_allocators PRIVATE /W4)
else()
    target_compile_options(memory_allocators PRIVATE -Wall -Wextra -Wpedantic)
endif()
```

Листинг 5: CMakeLists.txt